

H2SMS Practicals Key

2026-09-07

Table of contents

Preface	1
Practical sessions	1
R setup	3
Getting started in R	3
RStudio Projects	3
RStudio Settings	4
R packages and libraries	5
R scripts (.R) vs Quarto documents (.qmd)	5
Writing R code	5
R Basics	7
Objects & Functions	7
Data Structures	7
Accessing data	9
Logic and relational statements	11
Plotting	12
1 Appendix	21
1.1 Probability	21
1.2 Univariate statistics	21
1.3 The Binomial distribution	22
1.4 The Poisson distribution	22
1.5 Z scores	23
1.6 Samples taken from a population	23
1.7 The t distribution	23
1.8 ANOVA	24
1.9 Regression & correlation	24

Preface



Practical sessions

In these practicals, you will apply concepts you learn throughout the H2 Statistics for Marine Science module. The questions throughout are useful for learning and revision, while producing appropriate graphics and correctly describing results are essential parts of the scientific process. Expertise in these core skills is essential to do well in future courses and scientific projects. H2 Statistics for Marine Science is truly one of the most important courses you'll take!

You will use datasets provided on Brightspace, datasets included in R, and datasets you simulate yourself.

Please read through the practical before the class. Each session is scheduled for 3.5h with a recommended break in the middle. You will work through a series of coding exercises and questions which are not assessed or marked, but may appear in the assessments. You should complete all the material in each practical.

Each week, you are asked to complete a short problem set available on Brightspace and submit a rendered .html. Solutions will be posted the following week.

The practicals are provided [online](#) and as a pdf on Brightspace. The web version has additional features that are not possible with pdfs and is the recommended interface.

R setup

Getting started in R

R is a statistical language and computing platform that is widely used in the sciences. It is free and open source. We will be using it extensively. There are many resources available, including:

- Your notes and course material from H1 Maths and Data Science (also on Brightspace > Practicals)
- Courses such as those from [Software Carpentry](#) or [Swirl](#)
- Online videos such as [IQUIT R](#)

Use these resources as needed to complement, revise, and reinforce the concepts you'll learn during this course.

During the practicals, use 'copy-and-paste' thoughtfully. R is best learned through your fingers, and working through errors, though frustrating, is an essential skill.

RStudio Projects

Working in a 'Project' within RStudio is the best way to avoid working directory complications. This is a very common source of frustration and errors. It also helps organize work to ensure that necessary scripts, documents, data, and output are all contained within the same folder. Other benefits include tracking your history of R commands and integrating cleanly with more sophisticated version control (e.g., git – more to come later).

Create an R Project

We will create an R Project for the semester. During each practical, you should work within this project. To set up a new project as we need it:

1. Sign into OneDrive, then open RStudio
2. Select *File > New Project...*
3. Choose *New Directory > New Project*
4. Set the *Directory name* as **H2SMS** and use the *Browse* button by *Create project as a subdirectory of:* to set a convenient location in your OneDrive folder. Click 'Create Project'.
5. In the *Files* panel in RStudio, click the *New Folder* button and create a folder called **data**.
6. On Brightspace, download H2SMS_practicalData.xlsx in *Practicals* and move it into your newly created **data** folder.
7. Close RStudio (to learn how to open appropriately)

The code in the practicals assumes this organization. If you choose to put your data files elsewhere, you will need to update the scripts accordingly. You're now prepared and organized for a semester in R!

Opening R

When using a project, you should open R via the .proj file – *not* the script you plan to work on.

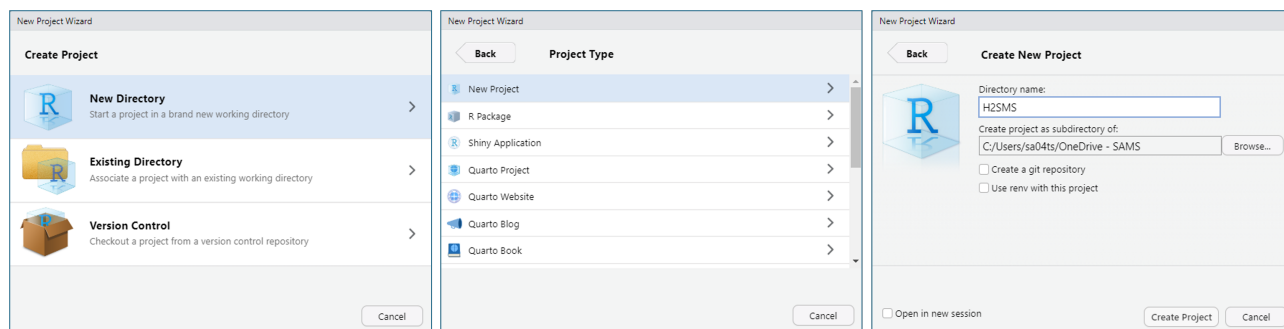


Figure 1: Setting up a new RStudio project.

🔥 I repeat: **Open H2SMS.Rproj** instead of the .R or .qmd file you plan to work on!
Opening the .R or .qmd from Windows Explorer will *not* set your R working directory correctly!

This is the start-up process you should use for the practicals:

1. Open Windows Explorer (or Finder on Mac) and find your **H2SMS** folder.
2. Double click on **H2SMS.Rproj**. This opens your R project with the working directory set to that folder. The working directory is shown at the top of the Console pane, and you can check it with `getwd()`. This is where R is ‘situated’ when loading or saving files.
3. Create a new .R file for that week’s practical.

To create a new .R file, click *New Blank File* at the top of the **Files** panel and selecting *R Script*. Name it *Practical_1.R* or similar.

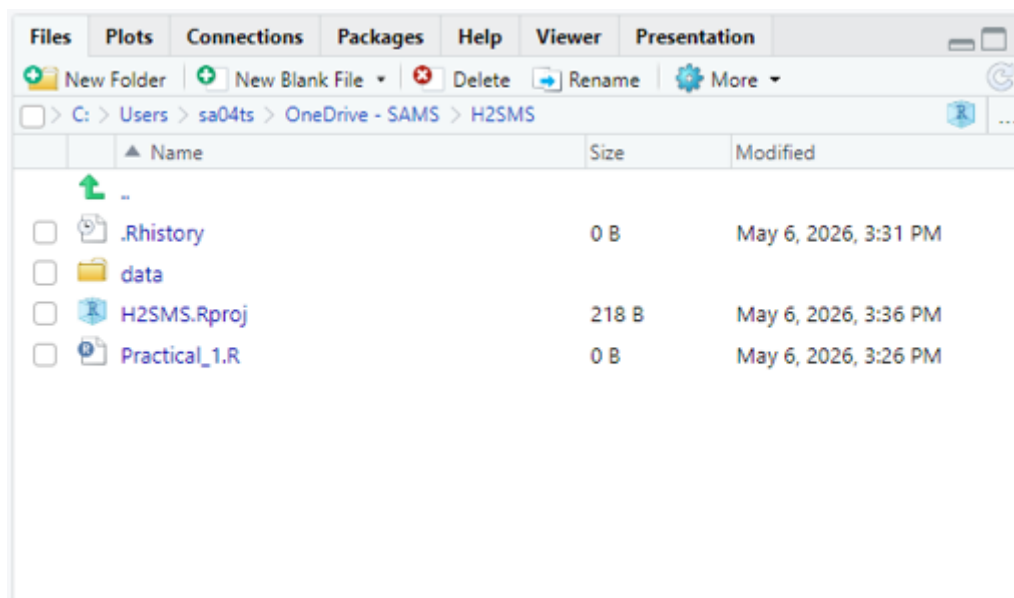


Figure 2: H2SMS directory structure.

RStudio Settings

You can adjust many settings in RStudio via Tools > Global options. In the Appearance tab in the popup box, you can set the theme (e.g., if you prefer a **dark theme**), font size, etc. The Code tab has many nice features as well (e.g., **rainbow parentheses** under Display).

R packages and libraries

R packages are collections of functions, custom data structures, and datasets that are developed by the user base. A new installation of R includes many useful packages, visible on the ‘Packages’ tab in RStudio. There are many additional packages available from the official CRAN repository or less officially from GitHub. If you find yourself re-using custom functions across projects, you can even create your own personal package.

To install a package from CRAN, use the function `install.packages("packageName")`. This downloads the package files to your computer. Each time you open R, you will need to load that package to use it with `library(packageName)`.

Installing from other package sources is slightly more complicated, so see me if you have a need.

View an overview of a package with `?packageName`, and then see a list of all of the functions by scrolling to the bottom of the help page and clicking the “index” link.

The help for each function is available with `?functionName`, and you can see the underlying code by running `functionName` without parentheses.

R scripts (.R) vs Quarto documents (.qmd)

In H1, you used R scripts (.R). These are just text files. The extension tells your computer to associate them with R, and also lets you run lines of code with ‘ctrl + enter’ and other convenient things in R and RStudio.

[Quarto documents](#) (.qmd) are also text files. However, RStudio interprets the text differently, allowing you to intersperse written prose, figures, references, R code, and output in a single document (similar to a jupyter notebook or a live script in Matlab). Code is marked as “chunks” and you can specify options for how the code and output are displayed. The text uses [markdown formatting](#), which allows all sorts of formatting (headers, bold, italic, equations, hyperlinks, tables, images...). RStudio can render a .qmd file into many other formats (e.g., pdf, docx, html, epub...).

Code chunks can be added with the green button with a ‘(+)c’ on the top right of a .qmd document in RStudio. Code chunks look like this:

```
```${r}
This is a code chunk
a <- 1:3
a
```
```

```
[1] 1 2 3
```

Run the full block with the green ‘play’ button at the top right of the block. The output from the code is shown just below the block.

For code-heavy work, Quarto documents are a handy way to produce nicely formatted output without the hassle of copying and pasting code, output, and figures into, e.g., a word document. There are many [guides](#) and [tutorials](#) online.

This manual is written as a Quarto book. **Versions of the .qmd files for each practical are available on Brightspace** to make things easier for you. You should now have these downloaded and saved in your project directory.

Note that the project will be submitted as a .qmd file with a version rendered to html.

Writing R code

R has established best practices to make your meaning clear. Just like any language, you can write with your own system, but it’s easier for everyone to use standard conventions. See the [full style guide](#) for more.

A few key points:

- Use `<-` to *assign* a value to an object. You may see `=`, which works, but is not preferred.
- Use `#` to write a comment which R will ignore.
- Use spaces to make your code legible: `a <- c(1, 2, 3)`.
- Avoid spaces in column names or file names as these are a pain to work with.
- Use names for objects that are short, but descriptive.
- Limit the length of a line of code to about 80 characters.
- Usually, variables should be nouns and functions should be verbs.
- Run the line of code where your cursor is (or everything you've selected) with `ctrl + enter`

R Basics

```
library(tidyverse)
theme_set(theme_classic())
```

Objects & Functions

In R, information is stored as *objects*. For our purposes, there are three main structures for storing data. These vary in the number of dimensions and in how many types of variables they can store. We name objects and assign a data structure to that name. This takes the form:

```
# objectName <- dataStructure
```

Functions perform operations on objects. They have names, like objects, but the function name is followed by brackets: `function()`. Functions often have *arguments* that must be specified. For example, the function `mean()` must be provided with data to calculate the mean of.

i R will interpret any unquoted text followed by brackets as a function, and any unquoted text as an object name.

Data Structures

Vectors

Vectors are one dimensional and are made up of a sequence of *elements*. Each element stores one value. All values within a vector must be the same data type (e.g., `numeric`, `character`, `logical`, etc). There are several ways to create a vector.

Here are some numeric vectors:

```
vec_a <- c(1, 4, 6)
vec_b <- 12:15
vec_c <- seq(1, 15, by=2)
```

Here are some character and logical vectors:

```
vec_d <- c("Site A", "Site B")
vec_e <- c(TRUE, FALSE, FALSE)
```

There are several useful functions for identifying characteristics of a vector:

```
length(vec_a) # how many elements?
```

```
[1] 3
```

```
class(vec_b) # what type of data?
```

```
[1] "integer"
```

```
str(vec_e) # what is the structure?
```

```
logi [1:3] TRUE FALSE FALSE
```

All values within a vector must be the same type. If you try to mix data types, R will convert them to a single type.

```
vec_f <- c(1, 3.2, "magnesium")
str(vec_f)
```

```
chr [1:3] "1" "3.2" "magnesium"
```

Matrices

Matrices are similar to vectors, but are two dimensional. They have rows and columns. They can be created using the function `matrix()`.

```
mx_A <- matrix(1:8, nrow=4, ncol=2)
mx_A
```

```
      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```

```
str(mx_A)
```

```
int [1:4, 1:2] 1 2 3 4 5 6 7 8
```

```
mx_B <- matrix(c("A", "B", "C", "D"), nrow=2)
mx_B
```

```
      [,1] [,2]
[1,] "A"  "C"
[2,] "B"  "D"
```

```
str(mx_B)
```

```
chr [1:2, 1:2] "A" "B" "C" "D"
```

Like vectors, all data must be the same type.

Data frames

Data frames are the most commonly used data structure by most users. They are two dimensional, but each column can be a different data type. They can be created directly using `data.frame()` or by loading data from external sources with functions like `read.csv()` or `read_excel()`.

```
# columnName = vectorOfData
df_A <- data.frame(columnA=1:3,
                   columnB=c("A", "B", "C"),
                   columnC=c(T, F, F))
df_A
```

```
  columnA columnB columnC
1       1       A    TRUE
2       2       B   FALSE
3       3       C   FALSE
```

```
str(df_A)
```

```
'data.frame':  3 obs. of  3 variables:
 $ columnA: int  1 2 3
 $ columnB: chr  "A" "B" "C"
 $ columnC: logi  TRUE FALSE FALSE
```

i Matrices and data frames must be rectangular: all columns have the same number of rows.

Accessing data

We typically perform operations on specific parts of our data. For example, we might calculate the mean of one variable, create a plot of one variable against another, or simply wish to view particular data points. Accessing parts of our data is called *extracting* or *subsetting*, and the method depends on the data structure.

Vectors

R organizes information using indexed elements. Indexes identify which element is which, starting at 1 for the first, and counting upward.

To extract a particular element from a vector, we specify that element's index in square brackets.

```
vec_a
```

```
[1] 1 4 6
```

```
vec_a[1]
```

```
[1] 1
```

```
vec_a[3]
```

```
[1] 6
```

To extract multiple elements, we can use a vector of indexes

```
vec_a[2:3]
```

```
[1] 4 6
```

```
vec_a[c(1, 3)]
```

```
[1] 1 6
```

Matrices

Extracting values from matrices is identical as for vectors, but requires the row index and column index since there are two dimensions.

```
mx_A
```

```
      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```

```
mx_A[2, 1] # [rows, columns]
```

```
[1] 2
```

```
mx_A[1:2, 2]
```

```
[1] 5 6
```

If we do not specify the rows or the columns, R will return all of them.

```
mx_A[1, ] # row 1, do not specify columns
```

```
[1] 1 5
```

```
mx_A[, 1] # column 1, do not specify rows
```

```
[1] 1 2 3 4
```

Data frames

Square brackets work for data frames identically to matrices. An additional method for data frames is to use the column names to extract a single column. Columns are accessed using `$` with the syntax `dataframeName$columnName`.

```
df_A
```

```
  columnA columnB columnC
1        1      A    TRUE
2        2      B   FALSE
3        3      C   FALSE
```

```
df_A$columnA
```

```
[1] 1 2 3
```

```
df_A$columnB
```

```
[1] "A" "B" "C"
```

Note that this returns a vector with the column contents. These can be extracted as above.

```
df_A$columnB[2:3]
```

```
[1] "B" "C"
```

Logic and relational statements

We often wish to compare values. R has several operators (a type of function) for making comparisons that return TRUE or FALSE:

- == : equal to
- != : not equal to
- > : greater than
- >= : greater than or equal to
- < : less than
- <= : less than or equal to
- | : or

These can be used with any data structure.

```
vec_b
```

```
[1] 12 13 14 15
```

```
vec_b == 13
```

```
[1] FALSE TRUE FALSE FALSE
```

```
vec_c
```

```
[1] 1 3 5 7 9 11 13 15
```

```
vec_c <= 5
```

```
[1] TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

```
vec_d
```

```
[1] "Site A" "Site B"
```

```
vec_d == "Site A"
```

```
[1] TRUE FALSE
```

```
vec_d != "Site A"
```

```
[1] FALSE TRUE
```

These statements are often useful for subsetting data, since a logical vector (TRUE/FALSE) can be used inside square brackets to specify which elements should be extracted.

```
vec_c
```

```
[1] 1 3 5 7 9 11 13 15
```

```
vec_c <= 5
```

```
[1] TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

```
vec_c[vec_c <= 5]
```

```
[1] 1 3 5
```

And for 2D objects like data frames, we can use a logical vector to identify which rows we'd like, and then state which columns to keep.

```
df_A
```

| | columnA | columnB | columnC |
|---|---------|---------|---------|
| 1 | 1 | A | TRUE |
| 2 | 2 | B | FALSE |
| 3 | 3 | C | FALSE |

```
df_A$columnB[df_A$columnB == "C"]
```

```
[1] "C"
```

```
df_A[df_A$columnB == "C", ] # specify rows, do not specify columns
```

| | columnA | columnB | columnC |
|---|---------|---------|---------|
| 3 | 3 | C | FALSE |

```
df_A[df_A$columnB == "C", 1:2] # specify rows, specify columns
```

| | columnA | columnB |
|---|---------|---------|
| 3 | 3 | C |

i Note that these objects never actually changed because we never assigned the output (using <-). To store the output of an operation, you can overwrite the original object or create a new object:

```
vec_x <- 1:5
vec_x2 <- vec_x[3:5] # store as a new object; vec_x is unchanged
vec_x <- vec_x[4:5] # overwrite; vec_x is now length 2
```

Plotting

Scatter plots

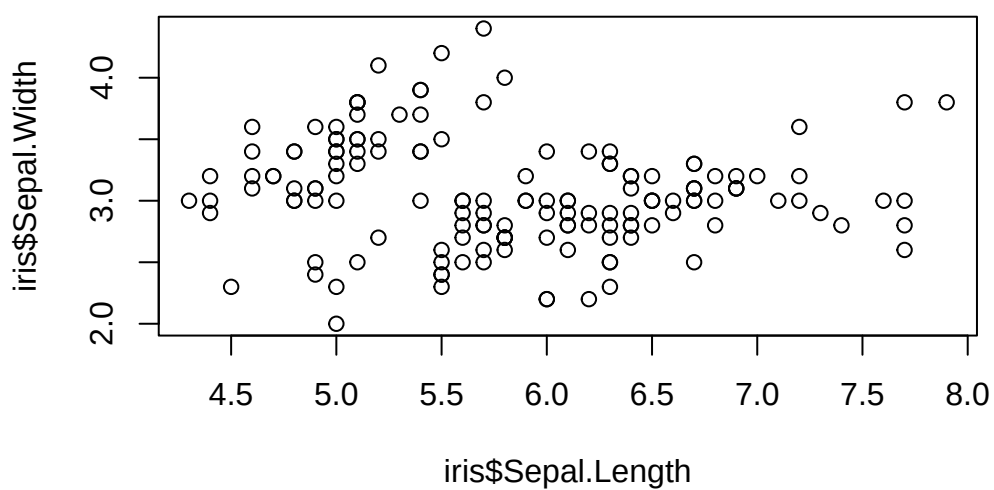
To make a scatter plot of two continuous variables, we can use either `plot()` or `ggplot()`. Further, there are two different approaches for `plot()`.

For illustration, we'll plot `Sepal.Length` on the x-axis and `Sepal.Width` on the y-axis.

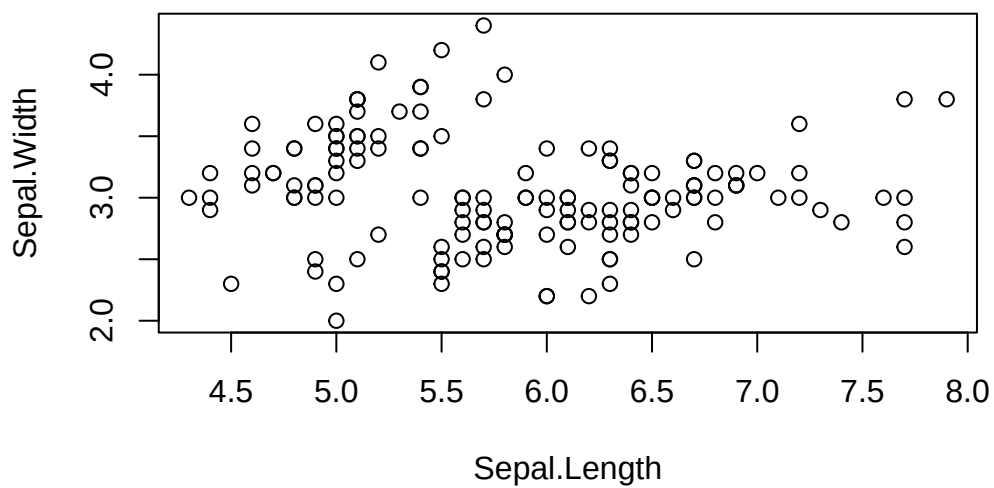
```
head(iris)
```

| | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---|--------------|-------------|--------------|-------------|---------|
| 1 | 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 2 | 4.9 | 3.0 | 1.4 | 0.2 | setosa |
| 3 | 4.7 | 3.2 | 1.3 | 0.2 | setosa |
| 4 | 4.6 | 3.1 | 1.5 | 0.2 | setosa |
| 5 | 5.0 | 3.6 | 1.4 | 0.2 | setosa |
| 6 | 5.4 | 3.9 | 1.7 | 0.4 | setosa |

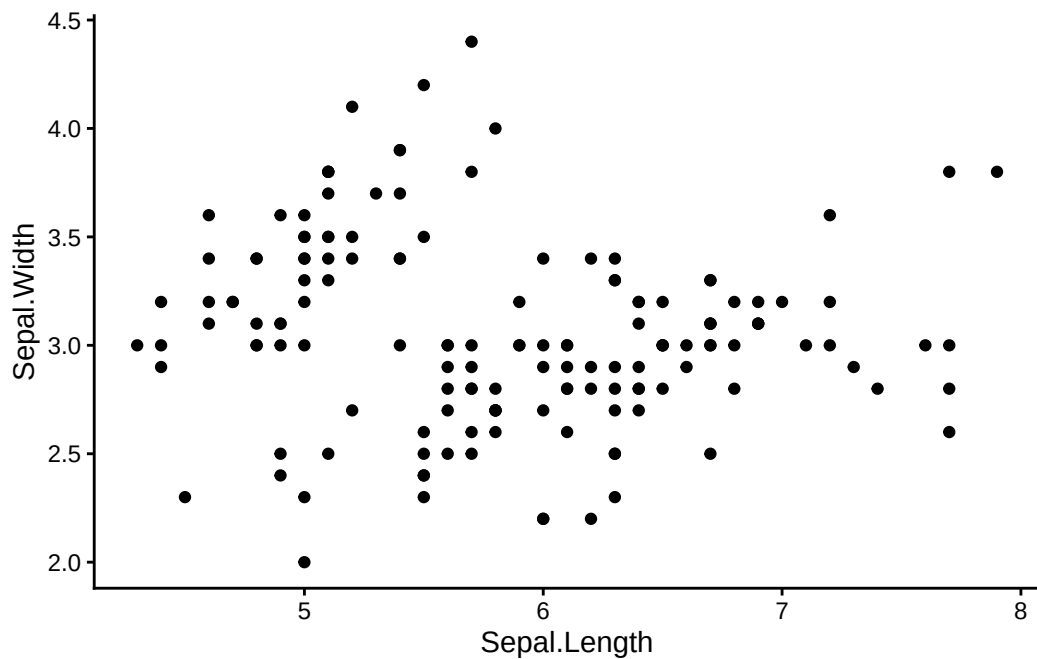
```
# plot(x_vector, y_vector)
plot(iris$Sepal.Length, iris$Sepal.Width)
```



```
# plot(y_column ~ x_column, data=dataframe_name)
plot(Sepal.Width ~ Sepal.Length, data=iris)
```



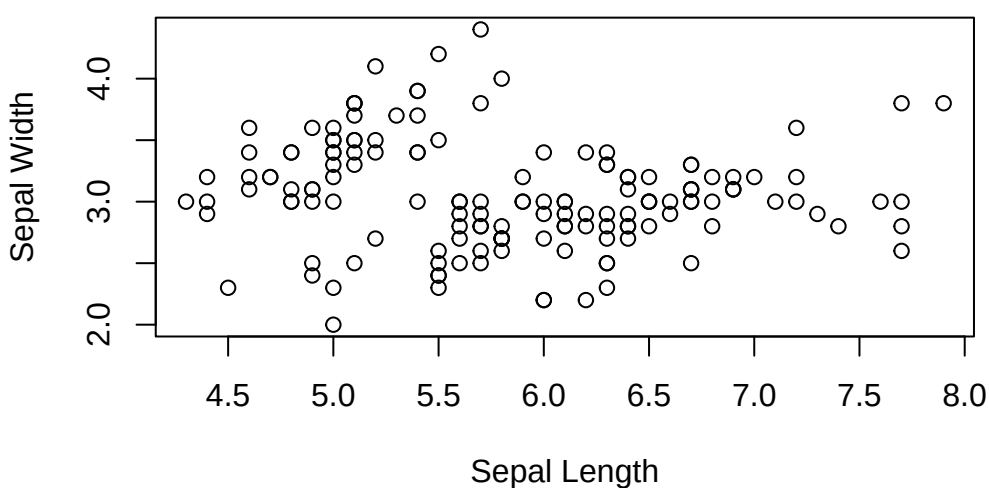
```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) +
  geom_point()
```



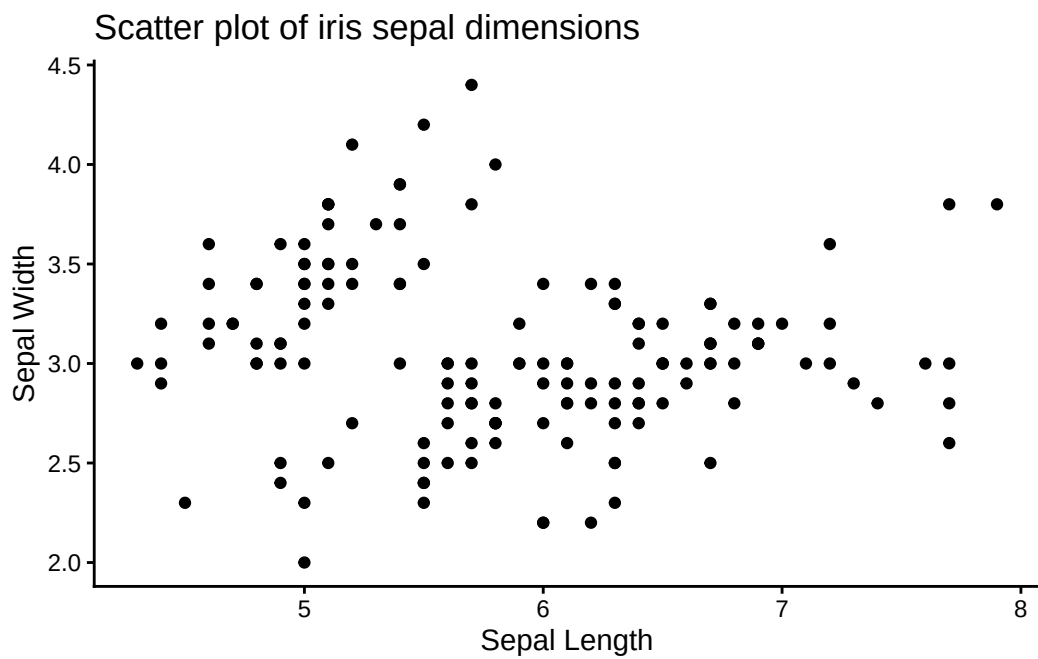
`plot()` is convenient for simple plots, but adding extra detail is often easier with `ggplot()`. Axis labels and titles can be added to each.

```
plot(Sepal.Width ~ Sepal.Length, data=iris,
     xlab="Sepal Length", ylab="Sepal Width",
     main="Scatter plot of iris sepal dimensions")
```

Scatter plot of iris sepal dimensions



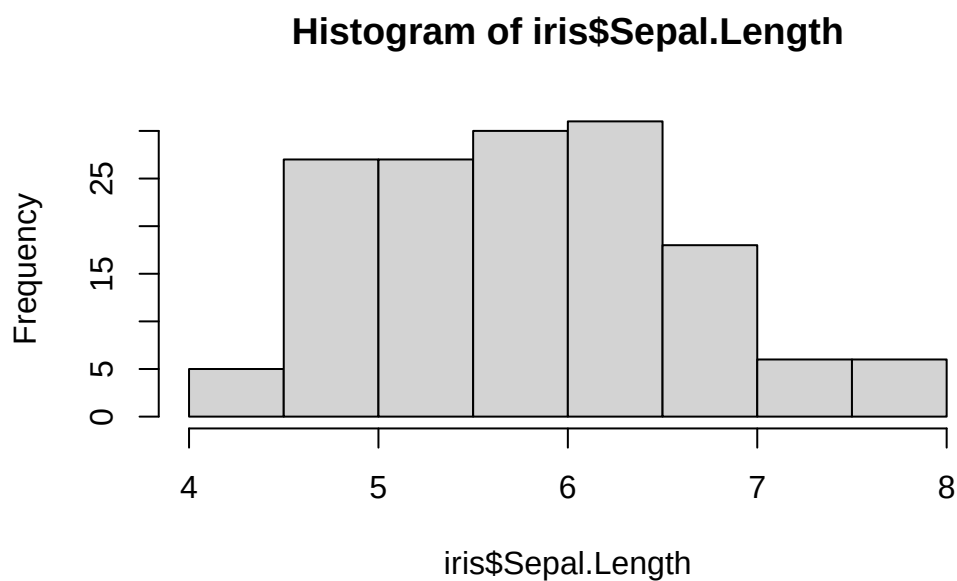
```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) +
  geom_point() +
  labs(x="Sepal Length",
       y="Sepal Width",
       title="Scatter plot of iris sepal dimensions")
```



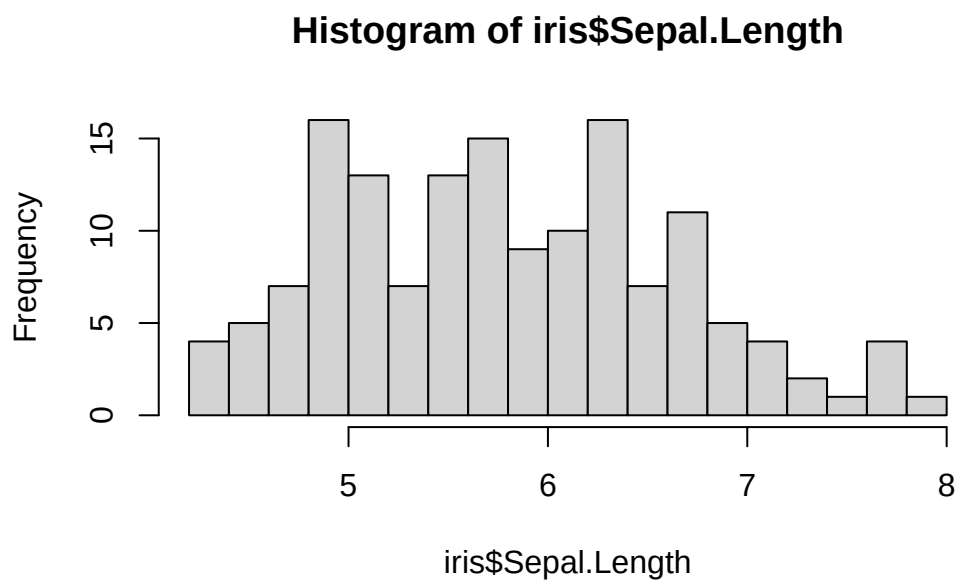
Histograms

Histograms are a common way to visualize the distribution of a continuous variable. We can use `hist()` or `ggplot()`. Note that these have different defaults for bin width, which should always be explored a bit anyway.

```
hist(iris$Sepal.Length)
```

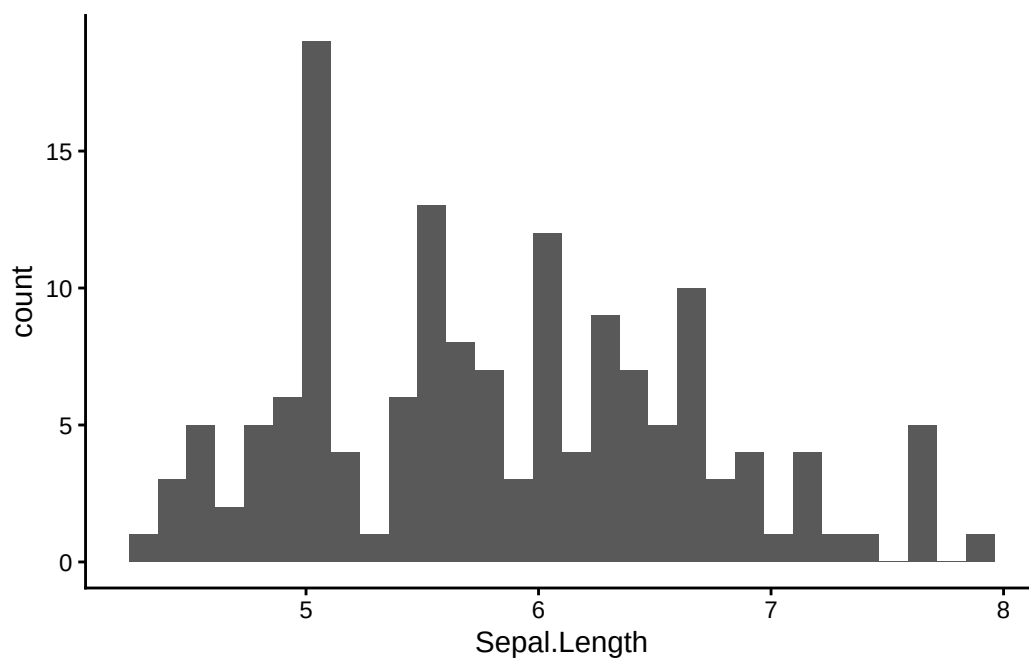


```
hist(iris$Sepal.Length, breaks=15)
```

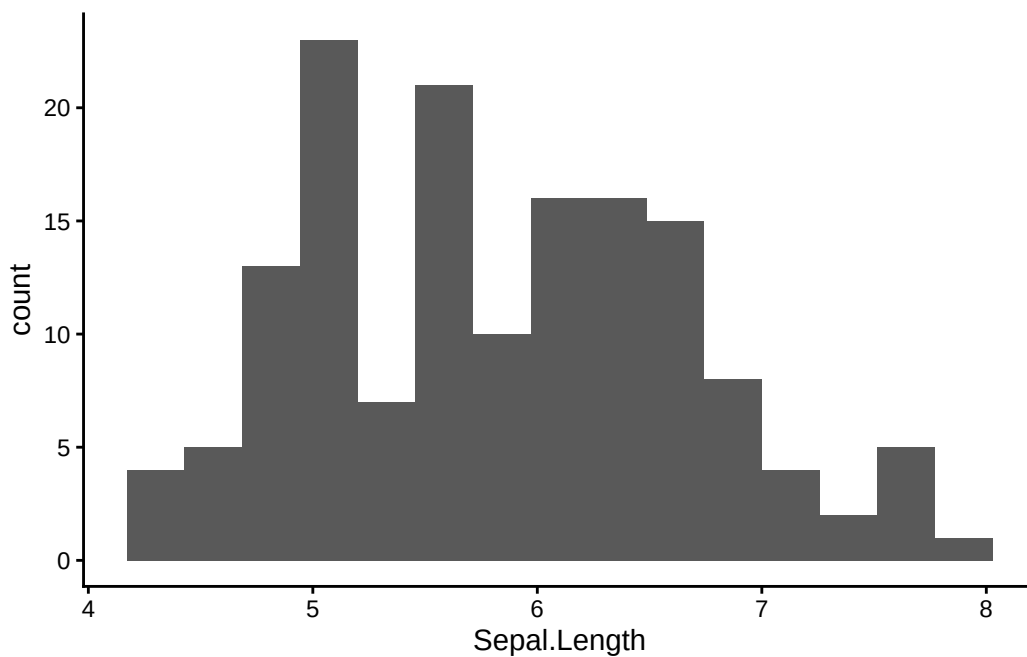


```
ggplot(iris, aes(Sepal.Length)) +  
  geom_histogram()
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.



```
ggplot(iris, aes(Sepal.Length)) +  
  geom_histogram(bins=15)
```

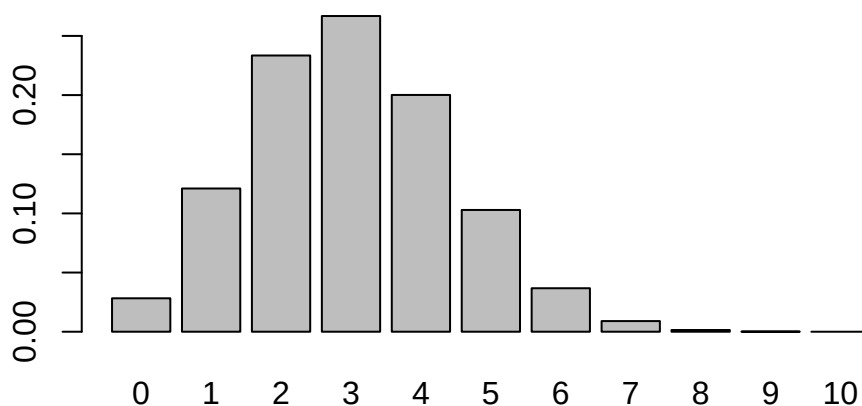


Bar plots

Bar plots visualize discrete distributions. They show the frequency of observations for each possible value. They are suited for discrete variables (e.g., counts), ordinal variables, or categorical variables.

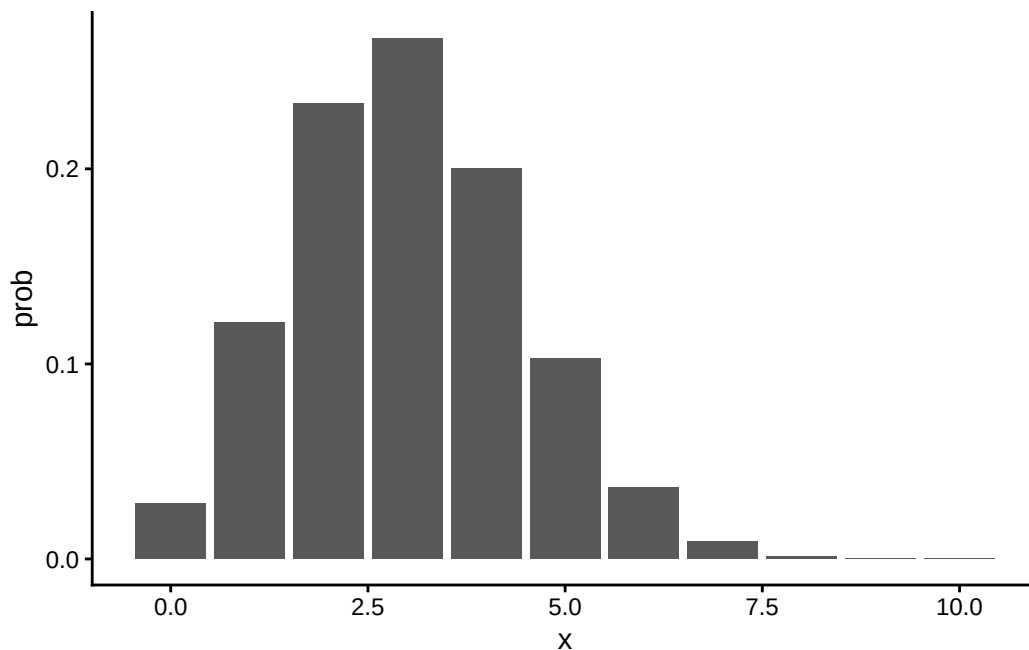
The `barplot()` function and `ggplot()` equivalent are somewhat different in this case. `barplot()` requires a vector of bar heights and an optional vector of names using the `names.arg=` argument.

```
binom_prob_df <- data.frame(x=0:10) |>
  mutate(prob=dbinom(x, size=10, prob=0.3))
barplot(binom_prob_df$prob, names.arg=binom_prob_df$x)
```



With `ggplot()`, we can use bar heights as above, but need to specify `stat="identity"` so that `geom_bar()` does not calculate any summary statistics. By default, `geom_bar()` automatically counts the number of observations.

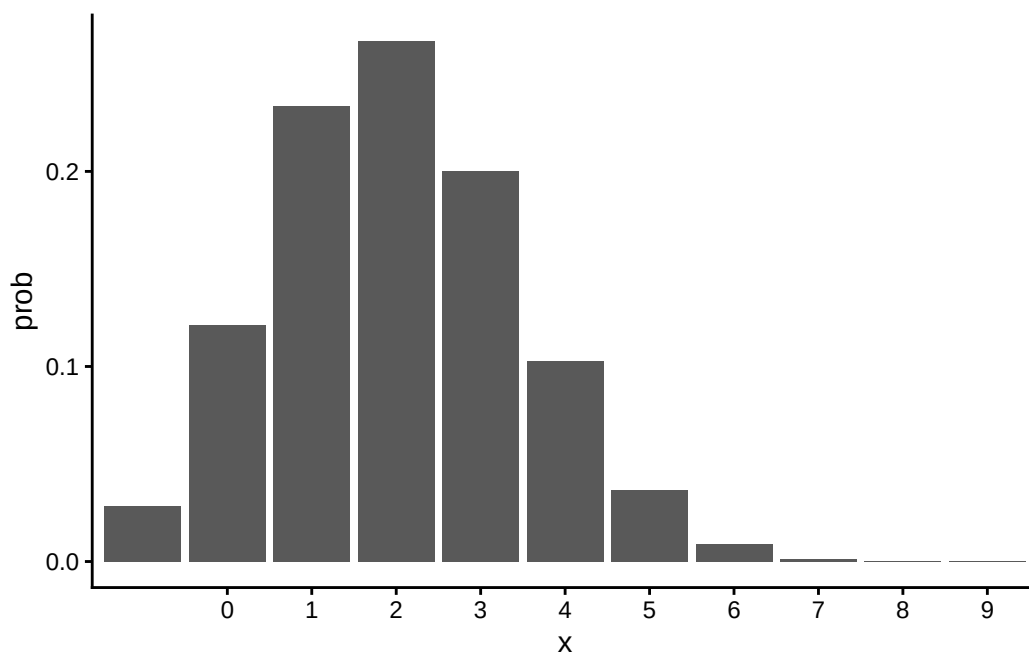
```
ggplot(binom_prob_df, aes(x=x, y=prob)) +
  geom_bar(stat="identity")
```



To make the x-axis a little prettier, we can add on an additional function.

```
ggplot(binom_prob_df, aes(x=x, y=prob)) +
  geom_bar(stat="identity") +
  scale_x_discrete(limits=0:10)
```

Warning in scale_x_discrete(limits = 0:10): Continuous limits supplied to discrete scale.
i Did you mean `limits = factor(...)` or `scale\_\*\_continuous()`?

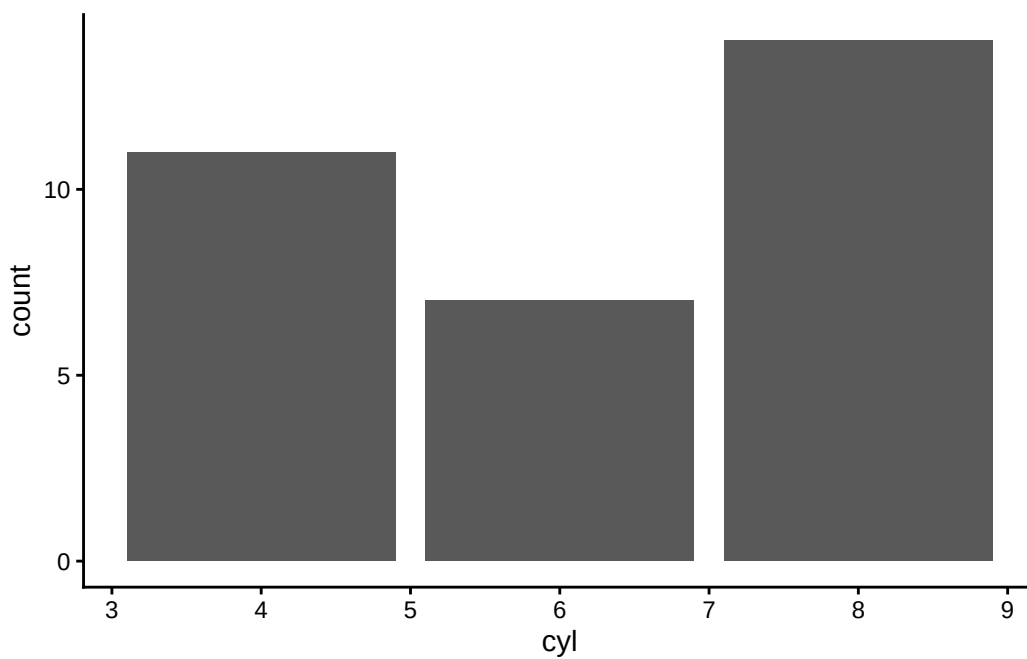


If we have recorded a suitable variable, `geom_bar()` can produce a plot of the frequencies:

```
head(mtcars)
```

| | mpg | cyl | disp | hp | drat | wt | qsec | vs | am | gear | carb |
|-------------------|------|-----|------|-----|------|-------|-------|----|----|------|------|
| Mazda RX4 | 21.0 | 6 | 160 | 110 | 3.90 | 2.620 | 16.46 | 0 | 1 | 4 | 4 |
| Mazda RX4 Wag | 21.0 | 6 | 160 | 110 | 3.90 | 2.875 | 17.02 | 0 | 1 | 4 | 4 |
| Datsun 710 | 22.8 | 4 | 108 | 93 | 3.85 | 2.320 | 18.61 | 1 | 1 | 4 | 1 |
| Hornet 4 Drive | 21.4 | 6 | 258 | 110 | 3.08 | 3.215 | 19.44 | 1 | 0 | 3 | 1 |
| Hornet Sportabout | 18.7 | 8 | 360 | 175 | 3.15 | 3.440 | 17.02 | 0 | 0 | 3 | 2 |
| Valiant | 18.1 | 6 | 225 | 105 | 2.76 | 3.460 | 20.22 | 1 | 0 | 3 | 1 |

```
ggplot(mtcars, aes(cyl)) +  
  geom_bar()
```



i There are many, many, *many* variations possible. See the tidyverse tutorial for a start, or explore the online documentation for [ggplot2](#).

Chapter 1

Appendix

1.1 Probability

1.1.1 Probability for equally likely outcomes

The probability of event A occurring is $P(A) = \frac{x}{n}$, where n is the number of trials and x is the number of trials during which A occurred.

1.1.2 Multiplication and addition

The general case of the multiplication rule is that, for two events A and B :

$P(A \& B) = P(A) * P(B|A)$, where $P(B|A)$ is the probability that B occurs given that A has already occurred.

The Addition Law states that, for two events A and B :

$$P(A \text{ or } B \text{ or } A \& B) = P(A) + P(B) - P(A \& B)$$

and

$$P(A \text{ or } B \text{ but not } A \& B) = P(A) + P(B) - 2 * P(A \& B)$$

and when A and B are mutually exclusive, then:

$$P(A \text{ or } B) = P(A) + P(B)$$

1.1.3 Bayes' theorem

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}, \text{ where } P(B) = P(A) * P(B|A) + P(A') * P(B|A').$$

1.2 Univariate statistics

1.2.1 Mean

`mean()`

The mean is calculated as $\bar{y} = \frac{\sum y}{n}$.

For measures of central tendency and dispersion, we use Greek letters to refer to population values and Latin letters to refer to samples:

| | Population | Sample |
|----------|------------|-----------|
| Mean | μ | $\bar{y}$ |
| Variance | σ^2 | s^2 |

| | Population | Sample |
|--------------------|------------|--------|
| Standard deviation | σ | s |

1.2.2 Median, quartiles and adjacent values

`median()`, `quantile()`, `IQR()`

With data ordered by rank, the median value is the middle value or the $(\frac{n+1}{2})^{th}$ value, the lower quartile, $Q1$, is the $(\frac{n+1}{4})^{th}$ value, the upper quartile, $Q3$, is the $(\frac{3*(n+1)}{4})^{th}$ value. The inter-quartile range is $IQR = Q3 - Q1$. The upper adjacent value is the upper value that is less than $Q3 + (1.5 * IQR)$. The lower adjacent value is the lower value that is more than the $Q1 - (1.5 * IQR)$. Outliers are defined as values that lie outside the range $Q1 - 1.5 * IQR$ or $Q3 + 1.5 * IQR$.

1.2.3 The sum of squares

The sum of squares is given by $SS = \sum y^2 - \frac{(\sum y)^2}{n}$ where n is the number of observations.

1.2.4 Measures of dispersion

`var()`, `sd()`, `length()`

When dealing with **populations** the following formulas are used:

Variance: $\sigma^2 = \frac{SS}{n}$

Standard deviation: $\sigma = \sqrt{\frac{SS}{n}} = \sqrt{\sigma^2}$

When dealing with **samples** the following formulas are used:

Variance: $s^2 = \frac{SS}{n-1}$

Standard deviation: $s = \sqrt{\frac{SS}{n-1}}$

1.3 The Binomial distribution

`dbinom()`, `pbinom()`, `qbinom()`, `rbinom()`

The binomial distribution describes the expected distribution for two mutually exclusive outcomes. The formula is given by:

$$P(x) = \frac{n!}{x!(n-x)!} p^x q^{n-x}$$

where $P(x)$ is the probability of x ‘successes’ occurring, n is the number of trials, p is the probability of x in a single trial, and q is the probability of *not* x in a single trial with $p + q = 1$.

A binomially distributed variable has **mean** $= n * p$ and **variance** $= n * p * q$.

1.4 The Poisson distribution

`dpois()`, `ppois()`, `qpois()`, `rpois()`

The Poisson distribution for a sample is defined as follows:

$$P(x) = \frac{\lambda^x e^{-\lambda}}{x!}$$

where λ is the mean and x is the value (count) of interest. Note that in a sample, we use the sample mean $\bar{y}$ as our estimate for λ

1.5 Z scores

`scale()`

The *standard normal distribution* is $Norm(\mu = 0, \sigma = 1)$. Z scores are used to convert any normal distribution to the standard normal distribution:

$$z = \frac{y - \mu}{\sigma}$$

where y is the value, μ is the mean of the population and σ is the standard deviation of the population.

1.6 Samples taken from a population

`sd()`, `sqrt()`, `length()`, `mean()`, `sqrt()`

1.6.1 Standard error of the mean

The *standard error* is the standard deviation of sample means:

$$SEM = \sigma_{\bar{y}} = \frac{\sigma}{\sqrt{n}}$$

where σ is the population standard deviation and n is the sample size. If we are testing a sample taken from a population *of known population mean and population standard deviation* we use the formula:

$$z = \frac{\bar{y} - \mu}{\sigma_{\bar{y}}}$$

where $\bar{y}$ is a particular sample mean, μ is the population mean, and $\sigma_{\bar{y}}$ is the standard error of the mean.

1.7 The t distribution

`dt()`, `pt()`, `qt()`, `rt()`

The t-distribution describes the distribution of T statistics expected under the null hypothesis.

1.7.1 The one sample t-test

`t.test()`

The T statistic is calculated as:

$$T = \frac{\bar{y} - \mu}{s_{\bar{y}}}$$

Where $\bar{y}$ is the sample mean, μ is the hypothesized population mean, and $s_{\bar{y}}$ is the standard error such that $s_{\bar{y}} = \frac{s}{\sqrt{n}}$ with sample standard deviation s .

Use `t.test()`, or find the p-value associated with your T with `pt(T, df-1)`.

1.7.2 Confidence intervals for sample means

`t.test()`, `qt()`, `mean()`, `sd()`, `length()`, `sqrt()`

The 95% CI for a mean is calculated as:

$$\bar{y} \pm t_{\alpha/2, df=n-1} * s_{\bar{y}}$$

1.8 ANOVA

`anova(lm()), aov()`

An ANOVA table contains the following:

| Source of variation | Sum of Squares | Degrees of Freedom | Mean Square | F |
|---------------------|----------------|--------------------|-----------------------------------|----------------------------------|
| Between groups | SS_{groups} | df_{groups} | $\frac{SS_{groups}}{df_{groups}}$ | $\frac{MS_{groups}}{MS_{error}}$ |
| Error | SS_{error} | df_{error} | $\frac{SS_{error}}{df_{error}}$ | |
| Total | SS_{total} | df_{total} | | |

with:

$$SS_{groups} = \sum n_j (\bar{y}_j - \bar{\bar{y}})^2$$

$$SS_{error} = \sum (y_{ij} - \bar{y}_j)^2$$

$$SS_{total} = SS_{groups} + SS_{error} = \sum (y_{ij} - \bar{\bar{y}})^2$$

where y_{ij} is the i^{th} observation in group j , $\bar{y}_j$ is the mean for group j , and $\bar{\bar{y}}$ is the grand mean. With N total observations, n_j observations per group, and k groups, the degrees of freedom are:

$$df_{groups} = k - 1$$

$$df_{error} = N - k = k(n_j - 1)$$

$$df_{total} = df_{groups} + df_{error} = N - 1$$

The critical value (CV) for samples of equal size is given as:

$$CV = q \sqrt{\frac{MS_{error}}{n}}$$

where q either comes from a table online (the *studentized range distribution*), or from `qtukey(1-alpha, k, df_error)`.

1.9 Regression & correlation

`lm(), cor(), confint(), predict()`

We do not bother calculating regression coefficients by hand.